

BUGMXT SI

Bug Triage Report

5-layer code integrity scan · ATANDA Command Centre · guide media + GitHub auto-sync (4b8f27a, 655657e)

BUGMXT SI — Bug Triage Report

Target: ATANDA Command Centre — commits 4b8f27a (guide-page walkthrough screenshots/GIFs) + 655657e (automatic GitHub mirror)

Engine: BUGMXT SI v1.0 (JCSE 46 / Platinum)

Model: claude-sonnet-4-6 (4-phase scan)

Scan duration: 148.7s total

Tokens: 130636 in / 6512 out

Generated: 2026-07-03T14:52:01.194Z

Executive Summary

Both change sets are structurally sound and passed typecheck, but the sync script contains two logic errors that can silently corrupt the mirror or break it permanently under reachable production conditions: a `-Z` diff parser that misreads deletion records (off-by-one field indexing when status and path are null-separated without a separating field gap), and an `isNonFastForwardError` regex that will fail to recognise GitHub's actual 422 message text, making the race-retry dead code. The capture script leaks one low-severity secret-handling risk: the `STAFF_ACCESS_CODE` is passed as a plain argument to `page.evaluate`, which is safe in-browser but the string is briefly resident in V8 heap in the Node host process and will appear in any heap dump or core dump taken during that call. No critical security defects were found in the guide-page rendering path; binary asset auditing is out of scope per activation context.

Layer 1 — Syntax Sweep

- **[SYNTAX-001]** | Severity: HIGH | File: `scripts/src/sync-github.ts` | Lines: 150–163 (`parseChanges`)

Issue: `git diff --name-status --no-renames -z` produces null-delimited output in the format `<status-letter>\0<path>\0<status-letter>\0<path>\0...`. Each record is **two** null-separated tokens: status then path. The parser advances `i += 2` per iteration and reads `parts[i]` (status) and `parts[i + 1]` (path), which is correct for Added/Modified entries. However, for **Deleted** files the status byte is `D` and the diff output is identical in structure — one token for the letter, one for the path — so that path itself is correct. The actual defect is the boundary check: the loop runs while `i < parts.length - 1`, which is correct, **but** the `filter(Boolean)` on the split will silently drop any empty string that results if the raw output ends with a double null (`\0\0`), shifting all subsequent indices by one and causing every subsequent path to be read as a status byte and vice versa. The guard should trim the raw string before splitting, not rely on `filter(Boolean)` mid-array.

Fix: Replace `raw.split("\0").filter(Boolean)` with `raw.trimEnd().split("\0")` so trailing null terminators do not produce empty strings that filter out and misalign the index walk.

- **[SYNTAX-002]** | Severity: MEDIUM | File: `scripts/src/sync-github.ts` | Lines: 55–70 (`getGithubToken`)

Issue: The `xReplLitToken` ternary has a logical short-circuit defect. It evaluates `REPL_IDENTITY` first; if

truthy it produces `"repl " + REPL_IDENTITY`. But `REPL_IDENTITY` in the Replit runtime is a signed JWT blob, not a bare token — the correct header format used by other connectors in this repo is different. More concretely, the `else` branch checks `WEB_REPL_RENEWAL` but this variable name does not appear in any Replit connector documentation or in the pattern cited as "copied from `lib/stripe.ts`" (which the diff does not include for verification). If both env vars are set to unexpected values, `xReplitToken` is set to a syntactically constructed string that will be rejected by the connector API with an opaque 4xx, and the thrown error message will not reveal which variable was the culprit. There is no unit of fallback.

Fix: Assert each env var individually and throw descriptive errors; do not concatenate env var values into auth header strings without validating their format first. Confidence note: `lib/stripe.ts` is not in the diff — the claim that this pattern is "copied" from it cannot be verified; flagging for manual cross-reference.

- **[SYNTAX-003]** | Severity: LOW | File: `scripts/src/capture-guide-shots.ts` | Line: 101

Issue: `await page.evaluate(() => (document as Document).fonts?.ready)` — the `as Document` cast is redundant inside a `page.evaluate` callback because the browser context already types `document as Document`. More critically, `fonts?.ready` returns a `Promise<FontFaceSet>`, but `page.evaluate` does not await the returned promise from within the serialised function body unless Puppeteer is told the callback returns a promise (which it is — Puppeteer does handle returned promises). However, the optional-chain `?.` means if `document.fonts` is undefined (non-standard environment) the evaluate resolves immediately with `undefined` and settle continues without waiting — the timeout guard below it (the `setTimeout(r, 2500)`) has already resolved, so this is a no-op safety net that silently becomes a no-op if fonts API is absent. Not a crash risk; flagging for clarity.

Fix: Use `await page.evaluate(() => document.fonts.ready)` without the optional chain, or add an explicit null guard that throws rather than silently skipping the font-ready wait.

- **[SYNTAX-004]** | Severity: LOW | File: `scripts/src/capture-guide-shots.ts` | Line: 64

Issue: `const { stdout } = await execFileAsync("which", ["chromium"])` — `which` exits with code 1 when the binary is not found, and `execFile` (promisified) rejects on non-zero exit. The rejection is not caught here; it propagates to the caller as an unhandled rejection with a `Node Error: Command failed` message that does not include the user-friendly hint message `"chromium not found on PATH; set CHROMIUM_BIN"`. The `if (!bin)` guard below is therefore unreachable on the failure path.

Fix: Wrap the `execFileAsync("which", ...)` call in a try/catch and throw the helpful message from the catch block, rather than relying on the `if (!bin)` guard that is never reached on failure.

Layer 2 — Logic & Outcome Audit

[LOGIC-001] | Severity: CRITICAL | ``scripts/src/sync-github.ts`` | `!`syncOnce()` / retry in `main()``

The retry after a non-fast-forward 422 calls `syncOnce(api, undefined, headSha)` with the **same** `headSha` the original run used`. If the winning concurrent sync advanced the remote to a *different* local HEAD (e.g. the workflow loop and the post-merge hook are on different commits), the retry re-reads the remote trailer correctly but still commits from the old `headSha`, producing a remote commit whose tree diverges from the current local state. The next run then diffs from the wrong base.

File: `scripts/src/sync-github.ts`, `main()` (~line 240)

Expected: retry syncs the current local HEAD.

Actual: retry always re-uses the pre-race `headSha`.

Fix: re-resolve `headSha` inside the retry path: `const headSha = ((await git(["rev-parse", "HEAD"])) as string).trim()` before the second `syncOnce` call.

Pathway: Patch

[LOGIC-002] | Severity: CRITICAL | `scripts/src/sync-github.ts` !' `parseChanges()`

--no-renames is passed but the -Z NUL-delimited parser assumes every record is exactly two NUL-separated tokens (status + path). With -Z and --name-status the format is <status>\0<path>\0, which is correct for A/M/D/T — but the loop strides `i += 2` and reads `parts[i+1]` without bounds-checking. If the final NUL-delimited pair is incomplete (empty trailing token after `.filter(Boolean)` removes the last `\0`), the last file is silently skipped and never synced or deleted.

File: `scripts/src/sync-github.ts`, `parseChanges()` (~line 120)

Expected: all changed files processed.

Actual: the last file in the diff may be silently dropped.

Fix: loop while `(i + 1 < parts.length)` and validate both tokens are non-empty before processing.

Pathway: Patch

[LOGIC-003] | Severity: HIGH | `scripts/src/sync-github.ts` !' `syncOnce()` — partial-upload, no cleanup

Blobs are uploaded to GitHub (POST `/git/blobs`) before the tree and commit are created. If the token expires or the process is killed after some blobs are uploaded but before the ref PATCH succeeds, those orphaned blobs persist on the remote indefinitely. More critically, the *next* run will re-upload all blobs from scratch (no blob SHA reuse), but the remote tree is still at the pre-run state with a trailer pointing to `fullBaseSha`. The diff will re-apply correctly, but the prior upload cost is wasted and the mid-run state is invisible to the operator.

File: `scripts/src/sync-github.ts`, `syncOnce()` (~lines 175–215)

Expected: idempotent re-run after partial failure is safe and efficient.

Actual: safe (re-runs correctly) but not efficient; no signal that a partial run occurred.

Fix: log a warning when blob count > 0 but ref PATCH is about to be attempted, so operators can detect orphaned-blob situations; consider storing uploaded blob SHAs locally for reuse on retry.

Pathway: Patch

[LOGIC-004] | Severity: HIGH | `scripts/src/sync-github.ts` !' `isNonFastForwardError()`

The 422-detection regex `Update is not` relies on GitHub's English error message prose, which is not part of a stable API contract. GitHub also returns 422 for other ref-update failures (e.g. branch protection violations, required status checks). A branch-protection 422 will trigger the one retry, which will fail again with the same 422, and that error is then **re-thrown with no distinguishing context** — the operator sees a confusing "retrying once" log followed by an unqualified failure, masking the real cause (protection rule) entirely.

File: `scripts/src/sync-github.ts`, `isNonFastForwardError()` and `main()` (~lines 230–248)

Expected: branch-protection failures surface a clear error immediately without consuming the single retry.

Actual: branch-protection 422 is silently treated as a race, consumes the retry, then throws a generic message.

Fix: parse the GitHub error body JSON field `message` for "non-fast-forward" (the canonical machine-readable string) rather than free-text matching.

Pathway: Patch

[LOGIC-005] | Severity: HIGH | `scripts/src/capture-guide-shots.ts` !' `captureGif()` — stale frame directory left on failed GIF encode

`captureGif` creates `/tmp/guide-gif-<id>/`, captures frames, then calls `ffmpeg`. If `ffmpeg` throws (codec missing, disk full), the `rm(framesDir)` cleanup at line ~153 is never reached and the frame directory persists. On a resumed run (without `--force`), `captureGif` returns early because `${id}.gif` does **not** exist, so the skip check is on the GIF not the frames. The next run therefore recreates the directory (`rm -rf` then `mkdir`) and proceeds correctly — but if the partial GIF **was** written (`ffmpeg` succeeded partially and wrote a truncated file), the skip check `await exists(gifPath)` returns `true` and the guide page embeds the truncated GIF.

File: `scripts/src/capture-guide-shots.ts`, `captureGif()` (~lines 110–155)

Expected: a failed capture leaves no partial output; the next run re-captures.

Actual: a truncated GIF written by a partially-succeeded `ffmpeg` is treated as complete and served.

Fix: write the GIF to a `.tmp` path, then `fs.rename` atomically on success; delete the `.tmp` in a `finally` block on failure.

Pathway: Patch

[LOGIC-006] | Severity: MEDIUM | `scripts/src/capture-guide-shots.ts`!`login()` — single login, no session validation before captures

`login()` is called once; the authenticated session cookie is then reused for all 13 routes with cumulative `settle()` delays totalling ~32 seconds plus navigation time. If the dev server's session TTL is shorter than the total capture duration (or the server restarts mid-run), subsequent `page.goto()` calls silently succeed (HTTP 200 on the unauthenticated redirect/login wall) and `captureStill/captureGif` write screenshots of the login gate rather than the authenticated page. The skip-existing logic then permanently caches those incorrect screenshots.

File: `scripts/src/capture-guide-shots.ts`, `main() / captureStill()` (~lines 160–180)

Expected: all captured screenshots show authenticated UI.

Actual: post-expiry captures show the login page and are permanently cached as valid.

Fix: after each `page.goto()` and `settle()`, assert a known authenticated DOM selector (e.g. the nav or a staff-only element) before calling `page.screenshot()`; throw if absent.

Pathway: Patch

Layer 3 — HARP (Human + AI Readability)

- **[HARP-AI-001]** `getGithubToken()` in `sync-github.ts` silently tries two different env-var shapes for the auth header (`REPL_IDENTITY !"repl "` prefix, `WEB_REPL_RENEWAL !"depl "` prefix) with no comment explaining the fallback semantics or which is expected in production. A future agent cannot determine which branch is the canonical path.

- **[HARP-AI-002]** `git()` returns `string | Buffer` and is cast with `as string` or `as Buffer` at every call site. The overload is enforced only by convention, not types — AI agents parsing the file will see the union return type and cannot statically verify safety. A typed overload signature would remove the ambiguity.

- **[HARP-AI-003]** `repoRoot()` is a zero-argument function that returns a hardcoded relative path computation (`"../.."` from `import.meta.url`). The name implies dynamic resolution but the logic is a constant. Naming it `getRepoRoot()` and adding a one-line docstring would prevent misread as a cached-value getter.

- **[HARP-HUMAN-001]** `settle()` in `capture-guide-shots.ts` contains `await page.evaluate(() => (document as Document).fonts?.ready)` — the `await` discards the Promise returned by `evaluate` wrapping a Promise (double async); the `fonts-ready` wait may silently no-op. The comment says "let fonts settle" but the mechanism is subtly broken, which will confuse the next maintainer.

- **[HARP-HUMAN-002]** `parseChanges()` iterates parts with stride 2 (`i += 2`) assuming `--name-status -z` always produces interleaved `STATUS\0PATH\0` pairs, but rename records (R/C) emit three tokens. The comment `// A, M, T` documents handled codes but not the deliberate `--no-renames` dependency that makes the assumption safe — the next reader cannot tell if the stride is correct without tracing the git flag.

Layer 4 — PFP (PDD Fidelity Protocol)

- **[PFP-DRIFT-001]** The `replit.md` contract states the sync loop retries **every 10 minutes** ("every 10 minutes via the always-on GitHub Sync workflow"). The `.replit` workflow shell command uses `sleep 600` (600 seconds = 10 minutes), consistent. However, `replit.md` also says "after every task merge (`scripts/post-merge.sh`)" — the post-merge hook calls `sync-github` before `cleanup-test-cost-data` completes, meaning a slow janitor run can block the merge-triggered sync for minutes, silently violating the "after every merge" guarantee.
- **[PFP-DRIFT-002]** The operating contract requires the connector token to **never be logged**. `getGithubToken()` throws `new Error("connector lookup failed: HTTP ${response.status}")` and `makeApi()` throws errors containing the full API path and up to 300 chars of response body. If GitHub returns the token in a 4xx error body (e.g. a redirect with `Location` containing a token fragment), it would surface in the caught-and-printed error message in `main()` via `console.error`. No scrubbing is applied.
- **[PFP-DRIFT-003]** The contract states **`force: false` on the ref PATCH is the invariant** — the mirror must never force-push. The retry path in `main()` calls `syncOnce(api, undefined, headSha)` with `baseArg = undefined`, which re-reads the remote trailer. If the remote head advanced to a commit that carries no `Replit-Commit: trailer` (e.g. a manual GitHub commit), the retry throws the "no trailer" error rather than force-pushing — correct — but this is not documented in the retry block comment, leaving a future agent uncertain whether the retry could ever produce a force-push.
- **[PFP-DRIFT-004]** `replit.md` documents the guide walkthrough's output directory as `artifacts/command-centre/public/guide-media/`. The diff writes captured assets to `attached_assets/guide/` and the commit message confirms this path. The import paths in `guide.tsx` reference `@assets/guide/...` which resolves via Vite alias. These paths are self-consistent internally but contradict the stated canonical path in the task description context ("writes them into `artifacts/command-centre/public/guide-media/`"), creating documentation drift a future agent would act on incorrectly.
- **[PFP-DRIFT-005]** The `replit.md` entry for `sync-github` states recovery requires `-- --base=<sha>` (double-dash separator for pnpm). `sync-github.ts` parses `process.argv.find((a) => a.startsWith("--base="))` which works whether or not pnpm passes the `--` separator, but the memory file `github-repo-mirror.md` documents recovery as `--base=<sha>` without the `--` separator. The inconsistency between the two doc surfaces will cause a future agent to supply the wrong invocation form.

Layer 5 — EAL Bayesian Triage

Severity scale: 5=CRITICAL, 4=HIGH, 3=MEDIUM, 2=LOW, 1=INFO. All other dimensions 1–5 (higher = worse/more likely/broader/harder to detect).

ID	Layer	Severity (1-5)					Likelihood (1-5)					Blast Radius (1-5)					Detectability (1-5)					EAL Score	Priority
LOGIC-001	2	5	4	4	5		**4.55**		CRITICAL														
LOGIC-002	2	5	4	4	4		**4.40**		CRITICAL														
LOGIC-004	2	4	4	3	5		**3.95**		CRITICAL														
SYNTAX-001	1	4	4	4	4		**3.95**		CRITICAL														
LOGIC-005	2	4	3	2	4		**3.30**		HIGH														
LOGIC-006	2	4	3	2	4		**3.30**		HIGH														
LOGIC-003	2	3	4	3	3		**3.20**		HIGH														
PFP-DRIFT-002	4	4	2	2	4		**3.00**		HIGH														
SYNTAX-002	1	3	3	3	4		**3.00**		HIGH														
PFP-DRIFT-003	4	3	3	3	4		**3.00**		HIGH														
PFP-DRIFT-001	4	2	4	2	3		**2.65**		MEDIUM														
PFP-DRIFT-004	4	2	3	3	3		**2.60**		MEDIUM														
HARP-AI-001	3	2	3	2	4		**2.50**		MEDIUM														
HARP-AI-002	3	2	3	2	4		**2.50**		MEDIUM														
PFP-DRIFT-005	4	2	3	2	4		**2.50**		MEDIUM														
HARP-HUMAN-002	3	2	3	2	3		**2.35**		MEDIUM														
SYNTAX-004	1	2	3	1	3		**2.20**		MEDIUM														
HARP-HUMAN-001	3	2	2	1	3		**1.95**		LOW														
HARP-AI-003	3	1	2	1	2		**1.45**		LOW														
SYNTAX-003	1	1	2	1	3		**1.50**		LOW														

> EAL Score = (Severity × 0.35) + (Likelihood × 0.30) + (Blast Radius × 0.20) + (Detectability × 0.15)

Bug Triage Board

1. [LOGIC-001] — Retry uses stale HEAD SHA after race loss

- **Severity:** CRITICAL
- **Blast Radius:** Entire mirror history — every subsequent sync diffs from a wrong base, silently accumulating divergence
- **Estimated Fix Time:** 15 minutes
- **Fix Pathway:** Patch
- **Regression Risk:** LOW — re-resolving HEAD inside the catch block is a one-line addition with no side effects
- **Why this matters:** When two syncs race, the losing retry commits whatever tree matched the old local HEAD, not the current one; the mirror's `Replit-Commit`: trailer then permanently anchors future diffs to the wrong commit, causing irreversible cumulative divergence without any error signal.

2. [LOGIC-002 + SYNTAX-001] — ``-z`` diff parser can silently drop or misindex files

- **Severity:** CRITICAL
- **Blast Radius:** Any run where the diff output ends with a trailing NUL — files are silently skipped or their status and path fields are swapped, causing deletions to be ignored and upserts to be missed
- **Estimated Fix Time:** 20 minutes
- **Fix Pathway:** Patch
- **Regression Risk:** LOW — replacing `filter(Boolean)` with `trimEnd()` + tightening the loop bound guard is isolated to `parseChanges()`
- **Why this matters:** Silent file-drop means changed or deleted files are never reflected on the remote, breaking the mirror invariant with no error; the bug is triggered by normal git output (NUL-terminated final record) and is therefore hit on every non-trivial sync.

3. [LOGIC-004] — 422 branch-protection failure misidentified as race, consuming the single retry

- **Severity:** CRITICAL (escalated from HIGH due to masking effect)

- **Blast Radius:** Any sync against a branch with protection rules — the retry path is consumed, the real error is obscured, and operators see misleading "concurrent run" logs before an opaque failure
- **Estimated Fix Time:** 30 minutes
- **Fix Pathway:** Patch
- **Regression Risk:** LOW — switch to matching the canonical GitHub JSON "message": "non-fast-forward" field instead of free-text prose; does not alter the happy path
- **Why this matters:** Branch protection violations produce an identical 422 status; swallowing the error as a false race means the operator cannot distinguish "someone enabled branch protection on GitHub" from a normal concurrent-sync event, delaying diagnosis indefinitely.

What architect review MISSED

1. **[LOGIC-001] — Retry HEAD staleness.** The architect review noted the race-retry pattern and flagged it as implemented ("the suggested race-retry was implemented"), but did not verify that `headSha` is re-resolved before the retry call. The retry passes the pre-race `headSha` verbatim, so the race fix is structurally present but functionally broken under the exact scenario it was designed for.
2. **[LOGIC-004] — 422 over-matching.** The architect approved `force: false` and the retry guard, but did not scrutinise what happens when a non-race 422 (branch protection, required status checks) fires. The `isNonFastForwardError` regex matches free-text prose rather than GitHub's stable machine-readable "non-fast-forward" JSON field, making it both fragile and over-broad — a class of permanent failures silently masquerades as transient race conditions.
3. **[LOGIC-005] — Truncated GIF treated as complete.** The commit message records that `account.png` was inspected for PII and found clean, but does not document any validation of GIF integrity. The partial-GIF scenario (ffmpeg writes a truncated file before crashing) is a silent correctness failure that permanently caches bad assets via the skip-existing logic — an edge case the architect's "authenticated headless capture confirms cards display correctly" check would not exercise.

GRO Operating State at end of scan

ADVISORY MODE — two CRITICAL logic errors in `sync-github.ts` (stale-HEAD retry, 422 misclassification) can silently corrupt the mirror or permanently mask configuration failures; human review and patching is required before the next production sync cycle.